

Care, connected.

From a healthcare problem to a working, privacy-first website.

DIGITAL MEDICO

Patient history. Medication adherence.

A direct line to the doctor.

24-HOUR HACKATHON

ROUND 2 • BUILD STORY

01 Emergency history

02 Doctoc
communication

03 Medication
reminders

Care loses context when records are scattered.

The problem in the original presentation became three concrete product requirements.

THE EMERGENCY GAP

Missing history

A doctor may need to act before the patient can explain allergies, medication or previous treatment.

THE COMMUNICATION GAP

Delayed answers

Patients wait for appointments to ask simple questions or clarify their care plan.

THE ADHERENCE GAP

Forgotten doses

A prescription is useful only when the patient can follow its schedule consistently.

Two portals. One connected care journey.

A shared backend connects the patient's control with the doctor's clinical actions.

PATIENT PORTAL

My health, my control

View medical history and prescriptions.

Ask questions and confirm doses.

Grant or revoke doctor access.

DOCTOR PORTAL

Care with full context

Select an assigned, consenting patient.

Update clinical records and prescriptions.

Review questions and dose activity.

THE RULE: shared care must never become unrestricted access.

How we built it, step by step.

A focused build sequence kept the prototype achievable and reviewable.

01

Define the scope

Read the deck; select core workflows.

02

Design the experience

Separate patient and doctor journeys.

03

Build authentication

Hash passwords; create secure sessions.

04

Enforce access

Check roles, patient identity and consent.

05

Connect medical records

Persist visits, profiles and prescriptions.

06

Add communication & alarms

Doctor replies and medication actions.

07

Validate the workflows

API, browser, mobile and build checks.

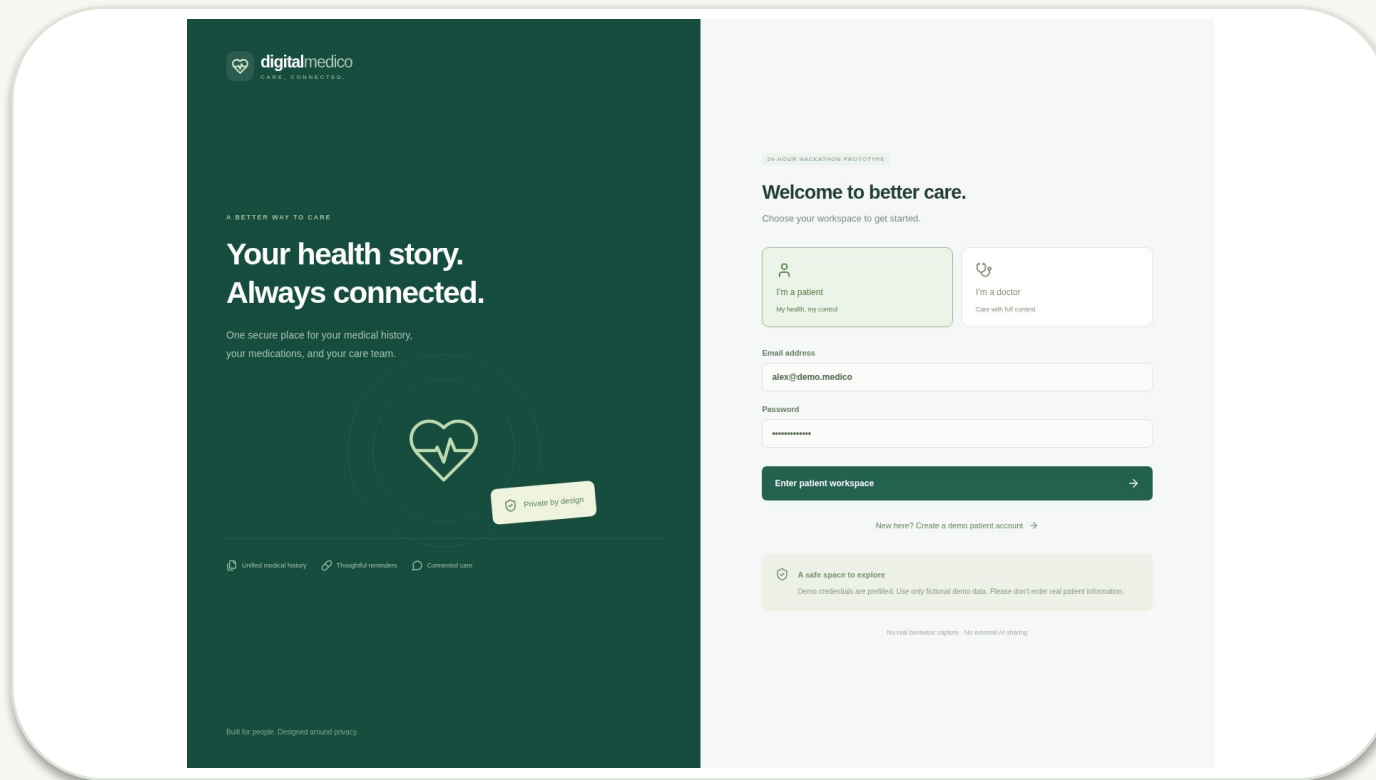
08

Package the website

VS Code setup and GitHub delivery.

First, make each portal trustworthy.

Authentication determines the workspace; the interface cannot grant itself a role.



A real sign-in flow

CHOOSE A PORTAL

Patient and doctor credentials are checked against their stored role.

VERIFY CREDENTIALS

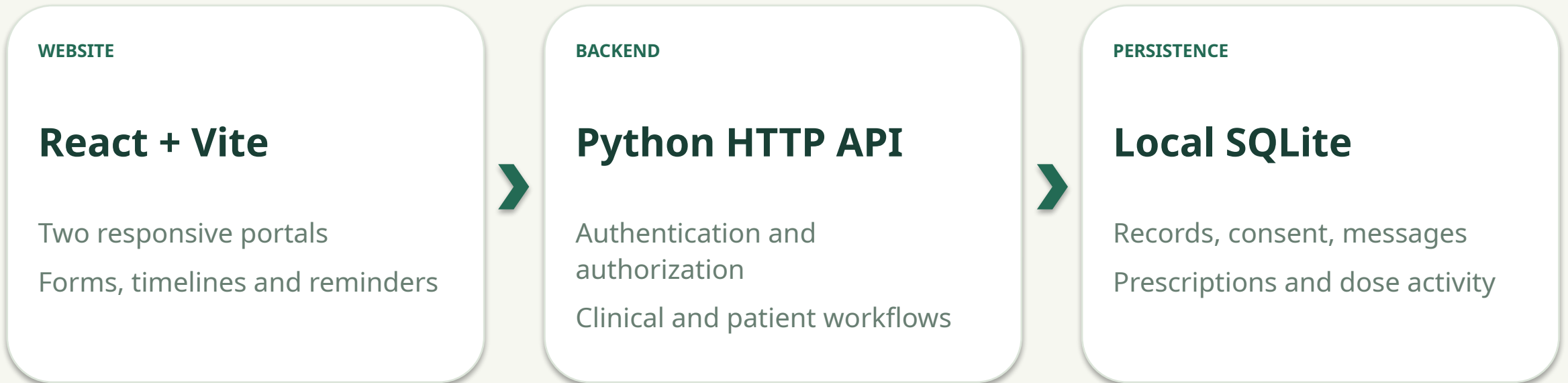
Normalized email + individually salted script password hashes.

RESTORE THE SESSION

HTTP-only cookie; refresh keeps an active session. Logout invalidates it.

Then, connect the interface to a real backend.

This is the architecture implemented in the current prototype.



Every medical request: session → role → patient assignment → consent → scoped query → audit.

Give every patient a separate health story.

We designed the data boundaries before adding more accounts.

IDENTITY

Users + patient profiles

Account role, assigned doctor, emergency ID.

PERMISSION

Consent grants

Explicit patient control over doctor access.

CLINICAL HISTORY

Records + prescriptions

Visits, clinical notes, medication instructions.

ADHERENCE

Dose schedules + logs

Today's status and preserved activity history.

COMMUNICATION

Messages + replies

Private questions, escalation and doctor responses.

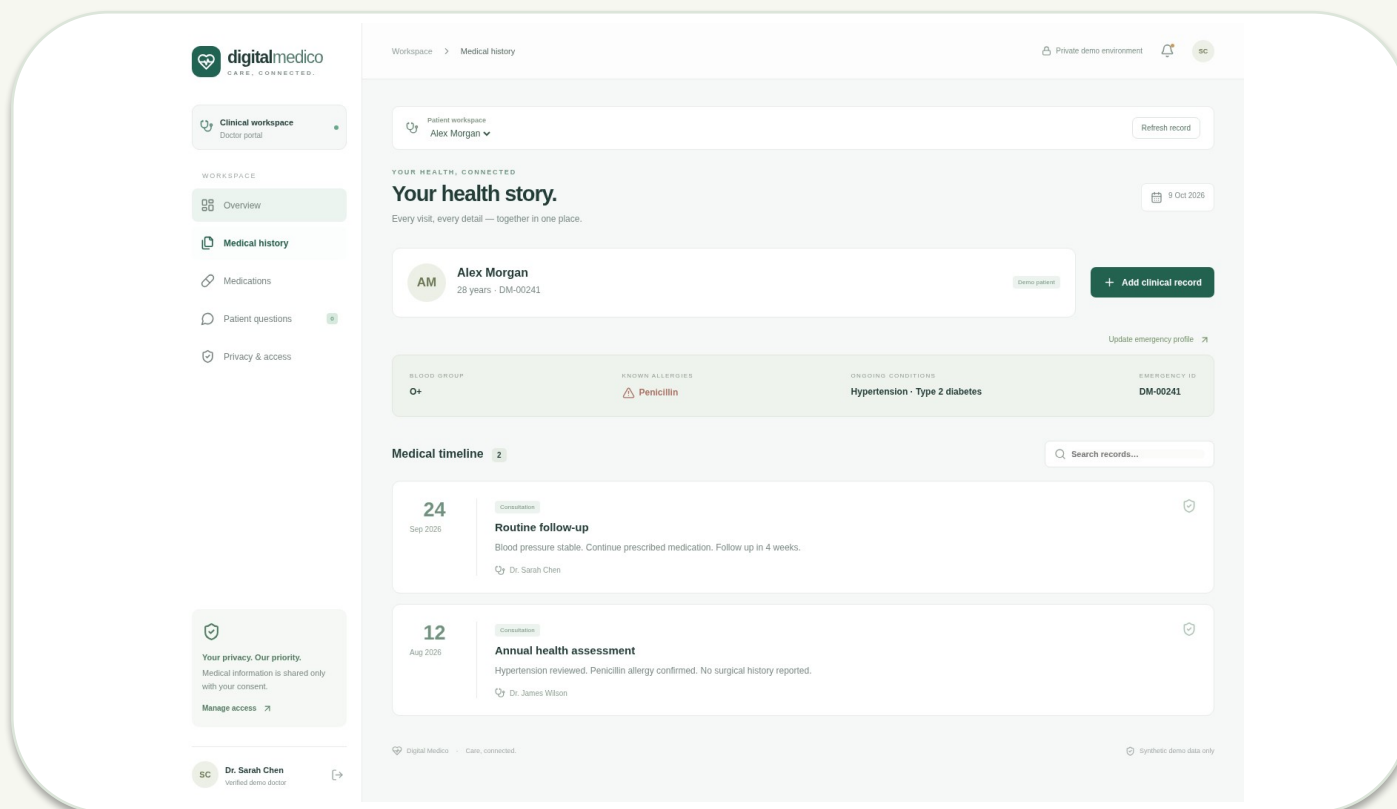
ACCOUNTABILITY

Audit events

Who accessed or changed which patient's information.

Turn a consultation into a lasting record.

Doctor updates become visible in the correct patient's portal.



Record once. Keep the context.

- 01 Select the consented patient
- 02 Review allergies and conditions
- 03 Add the visit and clinical notes
- 04 Save a prescription and dose time

VERIFIED: RECORDS PERSIST

Emergency history, with permission.

The demo verifies the workflow; biometric identity integration remains simulated.



No biometric data is captured. Revoked consent also blocks emergency access.
Identity verification and permission to read medical information are separate decisions.

Build Doctoc around safe escalation.

The assistant keeps the doctor responsible for clinical decisions.

PATIENT QUESTION → PERMITTED CONTEXT → GUIDANCE OR HUMAN REVIEW

SCHEDULE QUESTION

Read the prescription

Return this patient's recorded medication schedule. No invented dose or treatment.

CLINICAL QUESTION

Ask the doctor

Send uncertain questions to the clinician inbox; save and display the doctor's reply.

URGENT SYMPTOMS

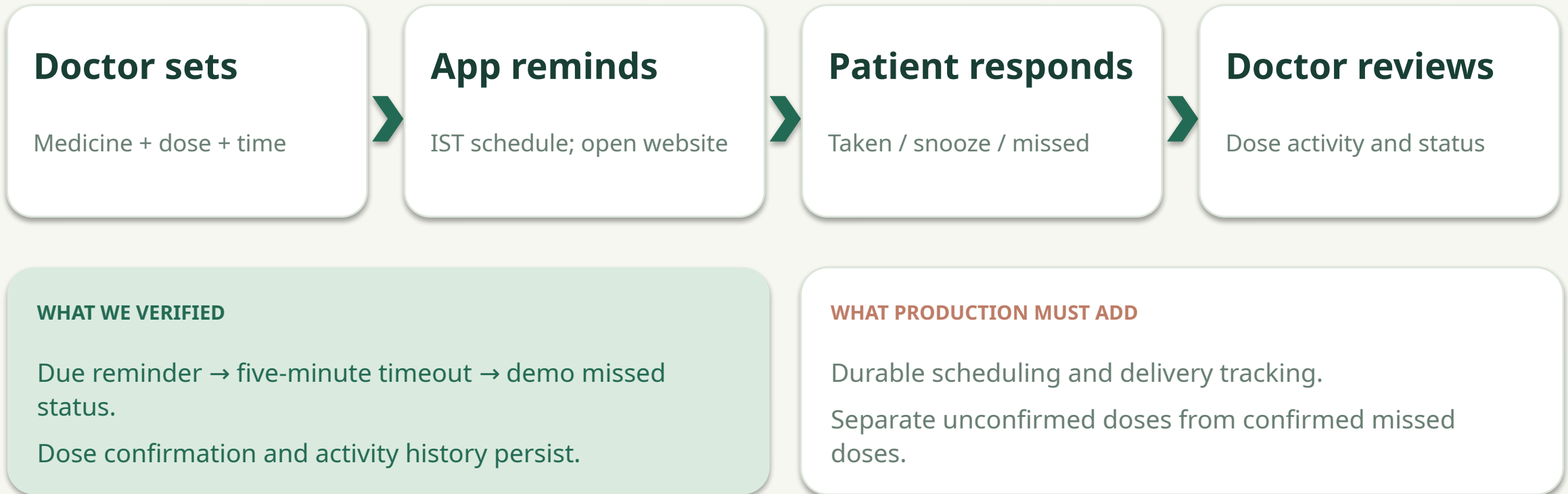
Seek urgent care

Show emergency-care guidance immediately. Never ask the patient to wait for chat.

Current Doctoc is scripted. No records are sent to an external AI. Without consent, questions stay private.

Make the prescription actionable every day.

Medication schedules, reminders and dose decisions are connected.



Privacy is a request boundary—not a badge.

We tested denial paths, not only successful screens.

Role protection

Patients cannot modify clinician-only records.

Patient isolation

Accounts cannot read or update another patient's data.

Consent enforcement

Revocation blocks doctor reads, edits and emergency access.

Interface cleanup

Open clinical forms and cached audit data clear after revocation.

Demo boundary: local storage is not encrypted at rest. Use fictional information only.

Prove the workflow before presenting it.

Verification covered the API, the browser, packaging and the optimized website build.

13

API tests passed

Authentication, roles, consent, patient isolation, profiles, prescriptions and messaging.

2

Portal journeys verified

Patient and doctor login, refresh restoration, wrong-portal rejection and logout.

PASS

Production build

Optimized frontend served with the API; packaged source installation also checked.

Browser flow: registration → patient selection → profile → record → prescription → reply → reminder → consent revocation.

Deliver a website the team can actually run.

Source code, VS Code tasks and a verified setup—not just screenshots.

OPEN THE PROJECT IN VS CODE

```
npm ci  
npm run dev
```

Open <http://localhost:5173>
Keep the terminal running.

FRONTEND

React source + responsive CSS

BACKEND

Python API + local database

DEVELOPMENT

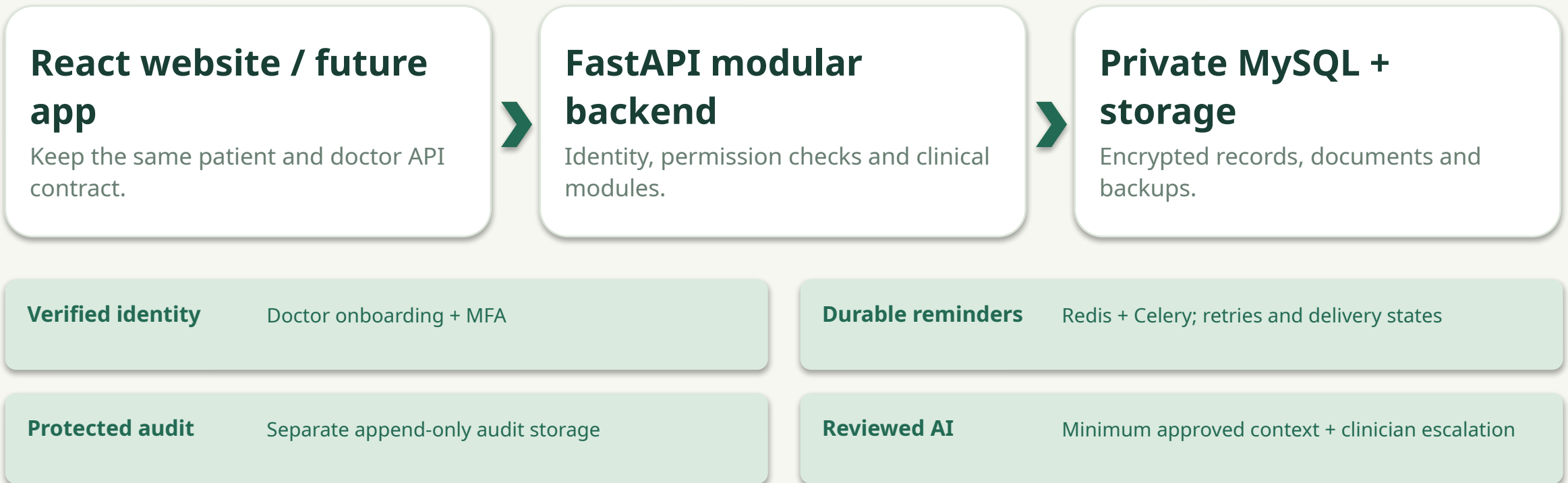
Node.js 22+ and Python 3.12+

HANDOFF

README, VS Code tasks and GitHub main

Scale the workflow. Upgrade the safeguards.

Proposed production architecture. These integrations are not implemented yet.



One demo. A complete care loop.

Show the judges the patient's control—not only the doctor's dashboard.

- | | | |
|----|---------|--|
| 01 | PATIENT | Read history, log a dose, send a question. |
| 02 | DOCTOR | Add care notes, prescribe, and reply. |
| 03 | PATIENT | See the update, then revoke access. |
| 04 | PROOF | Doctor access is denied; the audit explains why. |

Faster context. Consistent care. Patient-controlled access.